How I Built My Pokémon Card Binder (Without Knowing How to Code)

October 2026

The short version

I built my own app that tracks my Pokémon card collection. It runs on a computer in my house, and I open it on my phone from anywhere. I can't write code. An AI assistant wrote every line of it while I described what I wanted, tried each version, and said what to change.

It started as a spreadsheet of the cards I owned from one set. About a week and more than 75 small updates later, it does all of this:

- Shows every card in a set, in color if I own it and grey if I still need it
- Prices each card from recent real sales and keeps a price history
- Lays out a binder page by page, like a real binder, with a cover
- Searches every English card ever made and adds it to any binder
- Logs what I paid, keeps photos of my copies, and makes a shopping list
- Opens on a home page with a shelf of all my binders

The skill that mattered wasn't programming. It was knowing what I wanted, trying every version, and explaining clearly what was wrong or missing. If you can describe a problem to a friend, you can do this.

How it works, in one picture

Think of it as working with a builder who never gets tired:

1. **You describe** what you want, in normal words.
2. **The AI builds it** and tells you exactly what to click or type.
3. **You try it** and report what happened, good or bad.
4. **Repeat.** Each round makes it a little better.

You never have to understand the code. You only have to understand what you want the app to do, and notice when it doesn't.

What you need

Piece	What it does	What I used	Easiest choice for you
An AI assistant	Writes the code and walks you through every step	Claude	Claude. Free to try; a paid plan helps once the project gets bigger.
A computer	Runs your app	A home server that's always on	The laptop or desktop you already have, Windows or Mac
Python	The free program that runs the code the AI writes	Python	Python from python.org (the AI will walk you through it)
A private connection (later)	Lets your phone open the app when you're away from home	Tailscale	Tailscale, free for personal use

You don't need all four on day one. Start with the AI and your own computer. Add the rest when you want it.

Optional later: **GitHub**, a free website that stores your code and every past version of it. Mine uses GitHub so the app can update itself when I tap a button and confirm.

Step by step

Step 1: Set up the AI with a project

Make a free account at **claude.ai**. Then create a **Project**. A project is like a folder for one idea: it remembers your instructions every time you start a new chat inside it, so you don't have to explain yourself over and over.

Every project has a box for **instructions**. Paste in the project instructions from the prompt setup section below, filled in with your own details. This was the single most useful thing I did. It's how the AI knew I don't code, how I like things explained, and what rules my app follows.

Step 2: Describe your idea

Start a new chat inside the project and paste in the first message from the prompt setup, filled in with your idea. Keep the first version small. Mine was "show my cards and what they're worth." Everything else came later.

Step 3: Install Python (one time only)

The AI will tell you to do this, but here's what to expect:

1. Go to **python.org**, click **Downloads**, and download the version for your computer.
2. Open the file you downloaded and click through the installer.
3. **Windows only:** on the first screen, tick the box that says **Add python.exe to PATH** before you click Install. If you miss it, things won't work later. (If that happens, just tell the AI. It's a common fix.)

Step 4: Open the Terminal

The Terminal is a plain text window where you type commands. It looks scary, but you'll only ever copy and paste what the AI gives you.

- **Windows:** click Start, type **Terminal** (or **Command Prompt**), and press Enter.
- **Mac:** press Command and Space together, type **Terminal**, and press Enter.

When the AI gives you a command, copy it, paste it into the Terminal, and press Enter. Then copy whatever the Terminal shows back and paste it to the AI, even if it looks like gibberish. That text is how the AI sees what happened.

Step 5: Run your app and open it

The AI will give you a few files, tell you which folder to save them in, and give you one command to start the app. Then open your web browser (Chrome, Safari, Edge) and type the address it gives you, usually something like **http://localhost:8080**. Your app appears in the browser like a website, but it's running on your own computer.

Leave the Terminal window open while you use the app. Closing it stops the app.

Step 6: Try everything and report back

Click every button. Try it on your phone too (see Step 7). Then tell the AI:

- What worked
- What's broken (a screenshot is best)
- What's confusing
- One thing you'd like next

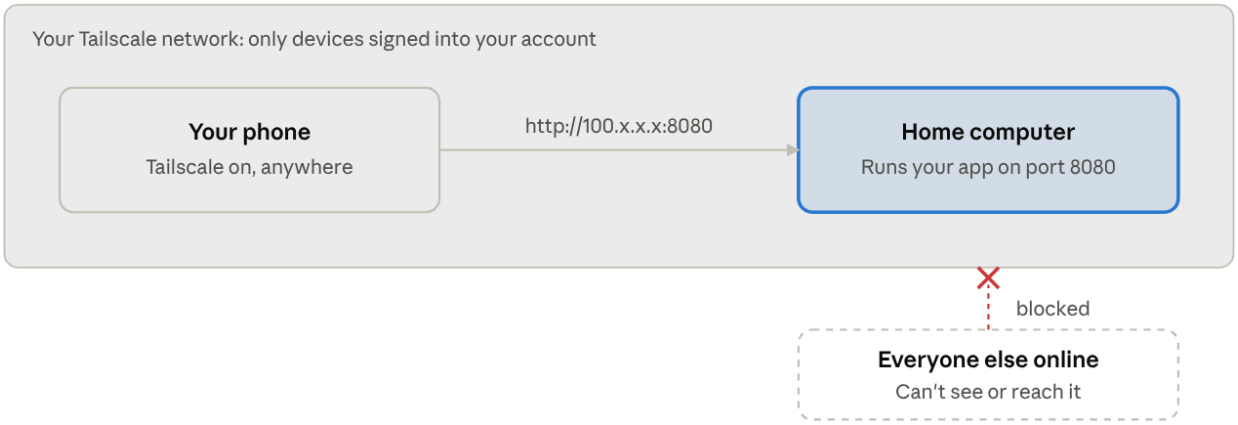
Ask for **one change at a time**. "Make cards I don't own grey." "Add a button that marks a card as owned." Small requests are easier to check and easier to undo.

Step 7: Open it on your phone

At home: your phone can open the app over your Wi-Fi. Ask the AI how to find your computer's address on your home network.

Away from home: use Tailscale. It makes your phone and your computer act like they're on the same Wi-Fi, even when you're miles away. Only devices signed into your Tailscale account can see each other, so your app stays private. The usual alternative, opening your home network to the internet, is risky and takes real know-how. Tailscale skips all of that.

Your phone reaches the app at home over your private Tailscale network



Setting it up (about 10 minutes):

1. Go to **tailscale.com** and sign up with a Google, Microsoft, or Apple account.
2. Install the Tailscale app on the computer running your app and sign in.
3. Install the Tailscale app on your phone and sign in with the same account.
4. In the Tailscale app, find your computer in the device list. It has an address that starts with **100.**, like 100.101.102.103.
5. On your phone, with Tailscale switched on, open the browser and type that address plus your app's number, like **http://100.101.102.103:8080**.

Type **http://** at the front. Some browsers otherwise try a secure connection the app doesn't have and show an error.

One catch: the app only works while that computer is on and awake. Later you can move it to an old laptop you leave running, or a home server like mine. Ask the AI to help when you're ready.

Step 8: Keep going

Every feature in my binder came from using it and thinking "it would be nice if..." Write those thoughts down and bring them to the AI one at a time.

The prompt setup

These are the exact kinds of messages that made my project work. Copy them, replace everything in [square brackets], and delete anything that doesn't apply to you.

1. Project instructions

Paste this into your project's instructions box once. The AI reads it at the start of every chat in the project.

```
PROJECT: [My app's name], a personal app that [what it does in one sentence,
like "tracks my Pokémon card collection and what each card is worth"].
It's just for me. [Any details that matter, like "I collect Unlimited, ungraded
cards" or "prices in US dollars".]

WHERE IT RUNS
- It runs on my own [Windows / Mac] computer and opens in a web browser.
- I want to open it on my phone too, privately (Tailscale, when we get there).
- My data (my collection, notes, photos) must stay on my computer, in its own
  folder, separate from the code, so an update can never erase it.
- Never put passwords, keys or personal details in the code.

HOW TO WORK WITH ME
- I don't know how to code and I'm new to computers. Use plain language and
  explain any technical word the first time you use it.
- Give me one step at a time, then stop and wait for me to say "done" or paste
  what I see. Never give me several steps at once.
- Tell me exactly where to click and exactly what to type or paste.
- When there are options, give me your one best choice plus one backup, not a
  long list.
- Before building something big, describe it or show me a mockup so I can say
  yes or no first.
- Test your changes before giving them to me, including on a phone-sized
  screen.
- Keep the look of the app consistent when adding new features.
- Suggest improvements on your own (speed, reliability, ease of use) and tell
  me why they help.
- Keep the app simple: as few files and add-ons as possible.

UPDATES
- Give each version a number (1.0, 1.1, 1.1.1). Small fixes get the last
  number, bigger changes get the middle number.
- Keep a simple list of what changed in each version, in plain words.
- Remind me to back up my data before any big change.
```

2. Your first message

Use this to start the first chat in the project.

```
I'd like to build [my app idea]. Here's what I want the first version to do:

- [The one most important thing, like "show every card in the Neo Destiny set"]
- [Second thing, like "let me mark which cards I own"]
- [Third thing, like "show what each card is worth from recent sales"]

[Optional: I've attached a spreadsheet / screenshot / photo of what I use now.]

Please start by telling me what I'll need and checking what's already on my
computer. Then walk me through it one step at a time.
```

If you have a spreadsheet, a photo, or a screenshot of something you like, attach it. Showing beats describing.

3. Messages for every day after that

Reporting a problem:

```
Something's wrong. Here's what I did: [the steps you took].
Here's what I expected: [what should have happened].
Here's what happened instead: [what you saw]. Screenshot attached.
```

Asking for something new:

```
I'd like the app to [what you want, like "show how much I've spent each month"].
Show me what it would look like before building it.
```

When you're stuck or lost:

```
I'm lost. Can you explain what we're doing and why, in simple terms,
and then give me just the next step?
```

When something looks off but you can't say why:

```
This part doesn't feel right [screenshot]. Can you give me two ideas to
make it better?
```

What made it work

- **Send screenshots.** A picture of what's wrong beats a paragraph describing it. On a phone, take a screenshot and attach it to your message.
- **Copy error messages exactly.** Paste the whole thing, even if it looks like nonsense. The AI reads it like a doctor reads an X-ray.
- **Report back after every step.** Say "done" or paste what the screen shows. The AI can't see your computer, so your report is its only way to know what happened.
- **Say what you want, not how to build it.** "I want to know how much I've spent each month" is enough. Working out the how is the AI's job.
- **Ask "why" whenever you're unsure.** That's how I learned what a server, Docker, and Tailscale even are.
- **Ask to see it first.** For anything bigger than a small fix, I asked for a mockup (a quick picture of the idea) before it was built. It's much easier to change a picture than a finished feature.
- **Back up before big changes.** Copy your app's data folder somewhere safe first, like a USB stick or cloud drive.
- **Never put passwords or keys in your code,** especially if you share it online. Ask the AI to check for you before sharing.
- **It's fine to change your mind.** My binder's cover went through six versions in one afternoon before I liked it.

When things go wrong

- **"Command not found" or "not recognized":** usually Python isn't installed yet, or (on Windows) the "Add to PATH" box wasn't ticked. Paste the message to the AI.
- **The page won't open:** check the Terminal is still open and the app is running. Make sure the address starts with **http://**.
- **It works on the computer but not the phone:** check both are on the same Wi-Fi, or that Tailscale is switched on, on both.
- **The AI's answer doesn't make sense:** say so. "I don't understand, can you explain it more simply?" always works.
- **A long chat starts going in circles:** start a new chat in the same project. The project instructions carry over, and a fresh start often helps.

Words you'll run into

Word	What it means
App	A program that does something useful. Mine is a website that only I can open.
Browser	The program you use to visit websites: Chrome, Safari, Edge.
Folder / file	Files are documents (like a photo or the app's code). Folders hold files.
Terminal	The text window where you type commands. The AI tells you exactly what to type.
Command	One line of text you paste into the Terminal and run by pressing Enter.
Python	A programming language. You install it once; it's what runs the code the AI writes.
Server	A computer whose job is to stay on and serve something, like your app, to other devices. An old laptop works fine.
Self-hosted	Running an app on your own computer instead of a company's.
localhost	"This computer." Opening http://localhost:8080 means "the app on this computer."
IP address	A device's address on a network, like 192.168.1.20 at home or 100.x.x.x on Tailscale.
Port	The number after the colon, like 8080. One computer can run several apps, each with its own number.
Tailscale	A free app that privately connects your phone to your computer from anywhere.
Version	A saved stage of your app, like 1.0 then 1.1. If a new one breaks, you can go back.
Backup	A copy of your data kept somewhere safe in case something goes wrong.
Mockup	A quick picture of an idea, made before the real thing is built.
Docker / container	A tidy box that holds an app and everything it needs. Handy on servers, but you can skip it when starting out.
GitHub	A free website that stores code and every past version of it. Useful for backups and updates.
Vibe coding	Building software by describing what you want to an AI instead of writing the code yourself.